



TRABAJO FINAL INTEGRADOR



Pipeline de Datos End-to-End en
Azure y Databricks para el análisis
de consumo de materiales de construcción

Diplomatura en Ingeniería de Datos 0226
Profesor: Antonio Gutierrez

Walter López

2026



Azure



Databricks



DELTA LAKE



Apache Spark



CSV



API / JSON

1 INDICE

2	Introducción	5
3	Objetivo del Proyecto.....	5
3.1	Problemática y Caso de Estudio	6
3.1.1	Problemática	6
3.2	Caso de Estudio	7
4	Situación Inicial	7
4.1	Información descentralizada.....	7
4.2	Problemas de calidad de datos	8
4.3	Procesos manuales	8
4.4	Falta de trazabilidad	8
4.5	Escalabilidad limitada	8
4.6	Solución Propuesta	9
4.7	Justificación Tecnológica	10
5	Ingesta y fuentes de los Datos	10
5.1	Introducción	10
5.2	Arquitectura de Ingesta.....	11
5.3	Origen de los Datos	11
5.3.1	Dataset de Consumo de Materiales (CSV)	11
5.3.2	Catálogo de Materiales (JSON)	12
5.3.3	Base de Datos de Proveedores (SQL)	12
5.4	Proceso de Ingesta	13
5.5	Uso del Archivo JSON en Reemplazo de API.....	13
5.5.1	Prompt Utilizado para la Generación de Datos	14
5.6	Pipeline de Azure Data Factory	16
5.7	Automatización del Pipeline de Ingesta	17



5.8	Capa RAW	19
5.9	Conversión y Estandarización de Archivos.....	19
5.9.1	Conversión de archivos CSV.....	19
5.9.2	Conversión de archivos JSON.....	19
5.9.3	Conversión desde Azure SQL Database a Parquet.....	20
5.10	Relación con Azure Databricks	20
6	Introducción a la Arquitectura Medallion.....	22
6.1.1	Capa Bronze	22
6.1.2	Capa Silver.....	22
6.1.3	Capa Gold.....	23
7	Capa Bronze	24
7.1	Introducción	24
7.2	Objetivos de la Capa Bronze.....	24
7.3	Origen de los Datos	24
7.4	Lectura de Archivos en Databricks	25
7.5	Lectura de materiales (JSON)	25
7.6	Lectura de proveedores (Parquet)	26
7.7	Conversión a Delta Lake	26
7.7.1	Escritura de Datos en Bronze.....	27
7.7.2	Beneficios de la Capa Bronze	28
7.7.3	Beneficios Operativos	28
7.8	Relación con las Capas Superiores.....	28
8	Capa Silver	30
8.1	Introducción	30
8.2	Objetivo de la Capa Silver	30
8.3	Lectura de Datos desde Bronze	30

8.4	Transformaciones Aplicadas	31
8.4.1	1. Integración de Datos mediante JOINS.....	31
8.4.2	2. Limpieza y Filtrado de Registros	32
8.4.3	3. Eliminación de Duplicados	33
8.4.4	4. Generación de Nuevas Columnas	34
8.4.5	Validaciones de Calidad de Datos.....	34
8.4.6	Manejo de Registros Inválidos	36
8.5	Persistencia de Datos en Silver	37
8.6	Importancia de la Capa Silver	38
8.7	Optimización de Tablas Delta con OPTIMIZE.....	39
8.7.1	Optimize	39
8.7.2	Uso de DESCRIBE HISTORY	40
9	Capa Gold	42
9.1	Introducción	42
9.2	Objetivo de la Capa Gold	42
9.3	Lectura de Datos desde Silver	42
9.4	Transformaciones Analíticas Aplicadas.....	43
9.4.1	1. Agregaciones por Obra	43
9.4.2	2. Generación de Ranking de Proveedores	45
9.4.3	3. Generación de Ranking de Materiales.....	47
9.5	Construcción de Tablas Analíticas	49
9.6	Optimización de Tablas Gold.....	50
9.6.1	Optimize	50
9.6.2	Auditoría y Trazabilidad con DESCRIBE HISTORY.....	50
9.7	Resultados Obtenidos	51
9.7.1	Importancia de la Capa Gold.....	51



10	Implementación del Pipeline Analítico en Azure Data Factory	53
10.1	Creación del Pipeline en Azure Data Factory	53
10.2	Configuración del Servicio Vinculado con Databricks	54
10.3	Configuración de los Notebooks	54
10.4	Ejecución Secuencial del Pipeline	55
10.5	Validación y Depuración del Pipeline	56
10.6	Publicación del Pipeline	57
10.7	Configuración del Trigger Automático	58
10.8	Relación entre Ingesta y Procesamiento	59
10.8.1	Pipeline de Ingesta (09:30 AM)	59
10.8.2	Pipeline Analítico (10:00 AM)	60
10.9	Monitoreo de Ejecuciones	60
10.10	Beneficios de la Orquestación Implementada	60
10.10.1	Beneficios Técnicos	60
10.10.2	Beneficios Operativos	61
11	Vista del contenedor Blob Storage	62
12	Compartir Usuario	62
13	Grupo de Recuso	63
14	Importante pipeline TFI_ingesta_InputRAW de Azure Database	63
15	Carpetas de Databrick	64
15.1	Link de Notebooks Medallion	64
16	Conclusiones y Reflexiones Finales	64



Informe – Trabajo Final Integrador

Pipeline de Datos End-to-End en Azure y Databricks para Análisis de Consumo de Materiales de Construcción

2 Introducción

El presente Trabajo Final Integrador tiene como objetivo el diseño e implementación de un pipeline de datos de punta a punta utilizando tecnologías cloud del ecosistema Microsoft Azure y procesamiento distribuido con Databricks.

La solución desarrollada permite integrar múltiples fuentes de datos heterogéneas relacionadas con el consumo de materiales de construcción, procesarlas bajo una arquitectura Medallion y generar tablas analíticas optimizadas para análisis y visualización.

El proyecto fue desarrollado utilizando:

- [Microsoft Azure](#)
- [Azure Data Factory](#)
- Azure Databricks
- [Delta Lake](#)
- [Apache Spark](#)
- [Azure Blob Storage](#)

3 Objetivo del Proyecto

El objetivo principal consiste en construir una arquitectura moderna de ingeniería de datos capaz de:

- Integrar datos provenientes de distintas fuentes.
- Automatizar procesos de ingesta y transformación.
- Garantizar calidad de datos mediante validaciones.
- Gestionar registros inválidos sin detener el pipeline.
- Implementar arquitectura Medallion (RAW → Bronze → Silver → Gold).
- Generar tablas analíticas para consumo posterior en herramientas de visualización y análisis de datos.

3.1 Problemática y Caso de Estudio

3.1.1 Problemática

En la industria de la construcción, la gestión de materiales representa uno de los factores más críticos dentro de la planificación y ejecución de obras. Cada proyecto constructivo requiere un seguimiento constante del consumo de materiales, costos asociados, proveedores involucrados y avance de las distintas etapas de obra.

Sin embargo, muchas empresas continúan trabajando con información dispersa en diferentes sistemas, archivos y formatos, lo que genera importantes dificultades operativas y analíticas.

Entre los principales problemas identificados se encuentran:

- Registros almacenados en múltiples formatos (CSV, bases de datos, APIs y planillas manuales).
- Falta de centralización de la información.
- Inconsistencias en los datos cargados.
- Duplicación de registros.
- Errores humanos en procesos manuales.
- Dificultad para obtener indicadores en tiempo real.
- Escasa trazabilidad de consumos y costos.
- Baja capacidad de análisis histórico.
- Procesos de reporting lentos y poco automatizados.

A medida que aumenta el volumen de información generado por las obras, los métodos tradicionales de administración comienzan a presentar limitaciones significativas. Esto impacta directamente en:

- El control presupuestario.
- La planificación de compras.
- El seguimiento de proveedores.
- La toma de decisiones gerenciales.
- La detección temprana de desvíos de costos.

Además, la incorporación de nuevas fuentes de datos, como APIs externas de precios o catálogos digitales de materiales, incrementa la complejidad técnica para integrar toda la información de manera consistente y confiable.

En este contexto surge la necesidad de implementar una solución moderna de ingeniería de datos que permita:

- Centralizar la información.
- Automatizar procesos.



- Garantizar calidad de datos.
- Escalar el procesamiento.
- Generar información analítica confiable.

3.2 Caso de Estudio

Empresa Constructora Ficticia: “Constructora Delta”

Para el desarrollo del presente trabajo se plantea el caso de estudio de una empresa ficticia denominada Constructora Delta, dedicada a obras civiles y proyectos de infraestructura.

La empresa administra simultáneamente múltiples obras de distinta magnitud, entre ellas:

- Edificios residenciales.
- Naves industriales.
- Obras viales.
- Remodelaciones comerciales.

Cada obra genera diariamente cientos o miles de registros relacionados con:

- Consumo de materiales.
- Solicitudes de compra.
- Ingreso de proveedores.
- Costos unitarios.
- Responsables de obra.
- Etapas constructivas.

4 Situación Inicial

Antes de implementar la solución propuesta, Constructora Delta presentaba los siguientes inconvenientes:

4.1 Información descentralizada

Los datos se encontraban distribuidos en:

- Archivos CSV exportados desde sistemas internos.
- Planillas Excel manuales.
- Bases de datos SQL de proveedores.
- APIs externas de materiales y precios.

Esto dificultaba la consolidación y análisis integral de la información.

4.2 Problemas de calidad de datos

Se detectaban frecuentemente:

- Registros duplicados.
- IDs inconsistentes.
- Campos vacíos.
- Errores de tipeo.
- Datos faltantes.
- Formatos incompatibles.

Por ejemplo:

- Cantidades negativas de materiales.
- Fechas inválidas.
- Materiales inexistentes.
- Proveedores sin identificación.

4.3 Procesos manuales

La generación de reportes era realizada manualmente por distintos sectores, lo que implicaba:

- Horas de trabajo operativo.
- Riesgo elevado de errores.
- Retrasos en la disponibilidad de información.
- Dificultad para actualizar indicadores.

4.4 Falta de trazabilidad

La empresa no contaba con un historial confiable de modificaciones ni versionado de datos, dificultando:

- Auditorías.
- Seguimiento histórico.
- Recuperación ante errores.

4.5 Escalabilidad limitada

El incremento del volumen de registros diarios provocaba problemas de rendimiento en los procesos tradicionales, especialmente en:

- Consultas masivas.

- Cruces de información.
- Procesamiento histórico.

4.6 Solución Propuesta

Para resolver esta problemática se diseñó una arquitectura de datos moderna basada en servicios cloud de Azure y procesamiento distribuido con Databricks.

La solución implementada permite:

- Centralizar todas las fuentes de datos.
- Automatizar la ingesta y transformación.
- Aplicar validaciones automáticas.
- Separar registros inválidos.
- Procesar grandes volúmenes de información.
- Generar tablas analíticas optimizadas.
- Ejecutar pipelines programados diariamente.

La arquitectura se basa en el modelo Medallion:

RAW → Bronze → Silver → Gold

Cada capa cumple una función específica dentro del ciclo de vida del dato.

4.6.1.1 Beneficios Esperados

Con la implementación del proyecto, Constructora Delta logra:

Beneficios Operativos

- Reducción de tareas manuales.
- Automatización de procesos.
- Mayor velocidad de procesamiento.
- Centralización de información.

4.6.1.2 Beneficios Técnicos

- Escalabilidad.
- Trazabilidad.
- Versionado de datos.
- Optimización de almacenamiento.

4.6.1.3 Beneficios Analíticos

- Indicadores confiables.

- Información consolidada.
- Análisis histórico.
- Mejor control presupuestario.
- Detección de desvíos de costos.

4.6.1.4 Beneficios Estratégicos

- Mejor toma de decisiones.
- Mayor capacidad de planificación.
- Seguimiento eficiente de proveedores.
- Visibilidad integral de las obras.

4.7 Justificación Tecnológica

La elección de las tecnologías utilizadas responde a necesidades específicas del proyecto:

Tecnología	Justificación
Azure Data Factory	Automatización y orquestación de pipelines
Azure Databricks	Procesamiento distribuido de grandes volúmenes
Delta Lake	Versionado, consistencia y optimización
Azure Blob Storage	Almacenamiento escalable y económico
Apache Spark	Transformaciones masivas de datos
SQL	Consultas analíticas y validaciones

5 Ingesta y fuentes de los Datos

5.1 Introducción

Uno de los aspectos fundamentales dentro de cualquier proyecto de ingeniería de datos es la correcta definición de las fuentes de información y los mecanismos de ingesta.

En el presente Trabajo Final Integrador se diseñó un proceso de ingesta orientado a integrar múltiples tipos de datos relacionados con el consumo de materiales de construcción.

La solución fue desarrollada considerando escenarios reales de empresas constructoras, donde la información generalmente proviene de distintas fuentes y formatos.

Las fuentes utilizadas en el proyecto fueron:

- Archivos CSV.
- Archivos JSON.

- Base de datos SQL.

Todos los datos fueron almacenados inicialmente en Azure Blob Storage para luego ser procesados mediante Azure Data Factory y Azure Databricks.

5.2 Arquitectura de Ingesta

La arquitectura de ingesta implementada sigue un enfoque desacoplado y escalable.

El flujo general del proceso es el siguiente:

Fuentes de Datos → Azure Blob Storage → Azure Data Factory → Azure Databricks → Delta Lake

Cada componente cumple una función específica:

Componente	Función
Azure Blob Storage	Almacenamiento centralizado de archivos
Azure Data Factory	Orquestación e ingesta
Azure Databricks	Transformaciones y procesamiento
Delta Lake	Persistencia optimizada y versionado

5.3 Origen de los Datos

5.3.1 Dataset de Consumo de Materiales (CSV)

El dataset principal del proyecto corresponde a los consumos de materiales registrados en distintas obras.

Este archivo fue generado en formato CSV simulando registros operativos reales de una empresa constructora.

Características del dataset

- Más de 500.000 registros.
- Información histórica de consumos.
- Datos asociados a obras y responsables.
- Registro de cantidades y unidades.
- Información temporal de cada consumo.

Campos principales

Campo	Descripción
consumo_id	Identificador único del consumo
material_id	Identificador del material
fecha	Fecha del consumo

obra	Obra asociada
etapa	Etapas constructiva
responsable	Responsable del consumo
unidad	Unidad de medida
cantidad	Cantidad consumida
costo_unitario	Costo por unidad
costo_total	Costo total calculado

5.3.2 Catálogo de Materiales (JSON)

Inicialmente se evaluó la posibilidad de utilizar una API externa para obtener información de materiales y precios. Sin embargo, debido a limitaciones de estabilidad, control de estructura y facilidad de integración académica, se decidió utilizar un archivo JSON estructurado.

Esta decisión permitió:

- Tener control total sobre el esquema.
- Simular una integración tipo API.
- Garantizar consistencia en los datos.
- Facilitar pruebas y validaciones.
- Evitar dependencias externas.

El archivo JSON representa un catálogo de materiales de construcción con información descriptiva y económica.

Campos principales del JSON

Campo	Descripción
material_id	Identificador del material
nombre_material	Nombre del material
categoria	Categoría del material
unidad_medida	Unidad utilizada
precio_unitario	Precio unitario
stock_minimo	Stock mínimo recomendado
proveedor_principal	Proveedor principal asociado

5.3.3 Base de Datos de Proveedores (SQL)

La información de proveedores fue almacenada en una base de datos SQL para simular un entorno empresarial real.

Esta fuente contiene información maestra de proveedores vinculados al abastecimiento de materiales.

Campos principales

Campo	Descripción
proveedor_id	Identificador del proveedor
nombre_proveedor	Nombre comercial
cuit	Identificación tributaria
ubicacion	Ciudad o provincia
telefono	Contacto
email	Correo electrónico
rubro	Especialidad del proveedor

5.4 Proceso de Ingesta

La ingesta fue orquestada mediante Azure Data Factory.

Se desarrollaron pipelines específicos para cada fuente de datos:

- Ingesta de CSV.
- Ingesta de JSON.
- Ingesta desde SQL.

Cada pipeline realiza:

1. Lectura de la fuente.
2. Copia hacia Blob Storage.
3. Validación básica de estructura.
4. Registro de ejecución.
5. Disparo del procesamiento en Databricks.

5.5 Uso del Archivo JSON en Reemplazo de API

Uno de los puntos importantes del proyecto fue la decisión de utilizar un archivo JSON estructurado en lugar de una API externa.

Desde el punto de vista conceptual, el JSON permitió simular un consumo típico de API REST, ya que:

- Mantiene una estructura semiestructurada.
- Puede ser procesado dinámicamente.
- Requiere parsing de objetos.
- Permite trabajar con esquemas flexibles.

Además, el uso de JSON facilitó:

- La integración en Azure Data Factory.
- El procesamiento distribuido en Spark.
- La inferencia de esquemas.
- La lectura incremental.

Esta estrategia permitió conservar la complejidad técnica de una integración moderna sin depender de servicios externos.

5.5.1 Prompt Utilizado para la Generación de Datos

Para la generación de los datasets sintéticos utilizados en el proyecto se empleó inteligencia artificial generativa con prompts específicos orientados a simular escenarios reales de construcción.

Prompt ejecutado para generación del JSON de materiales

Generar un archivo JSON con 200 materiales de construcción diferentes.

Cada material debe contener:

- material_id
- nombre_material
- categoria
- unidad_medida
- precio_unitario
- stock_minimo
- proveedor_principal

Los materiales deben representar productos reales utilizados en obras civiles y construcción.

Incluir categorías como:

- Cemento
- Hierro
- Arena
- Pintura
- Electricidad
- Sanitarios
- Revestimientos

- Aislaciones

Los datos deben tener formato consistente y realista.

Prompt ejecutado para generación del CSV de consumos

Generar un archivo CSV con más de 500000 registros simulando consumos de materiales de construcción.

El dataset debe incluir:

- consumo_id
- material_id
- fecha
- obra
- etapa
- responsable
- unidad
- cantidad
- costo_unitario
- costo_total

Los registros deben representar consumos diarios de distintas obras civiles.

Los datos deben mantener consistencia entre materiales, costos y unidades.

El archivo debe estar preparado para ser utilizado en pipelines de ingeniería de datos.

5.5.1.1 Beneficios de la Estrategia de Ingesta

La estrategia implementada permitió:

- Simular un entorno empresarial real.
- Integrar múltiples formatos de datos.
- Automatizar procesos de carga.
- Centralizar información.
- Escalar el procesamiento.
- Facilitar validaciones posteriores.
- Garantizar trazabilidad.

Además, el uso de Azure y Databricks permitió construir una arquitectura preparada para futuras extensiones, incluyendo integraciones reales mediante APIs, streaming o procesamiento near real-time.

Diseño de la Ingesta y Organización del Data Lake

Con el objetivo de construir una arquitectura ordenada, escalable y alineada con buenas prácticas de ingeniería de datos, se diseñó una estrategia de almacenamiento centralizada utilizando Azure Blob Storage como Data Lake principal del proyecto.

La lógica implementada consiste en separar claramente los datos de entrada de los datos procesados, permitiendo mantener trazabilidad, control de versiones y desacoplamiento entre las distintas etapas del pipeline.

Estructura del Blob Storage

Dentro del contenedor principal del Blob Storage se definieron distintas carpetas organizadas según el ciclo de vida del dato.

Carpeta Input

La carpeta input funciona como punto inicial de recepción de archivos.

En esta ubicación se cargan los datasets provenientes de las distintas fuentes:

- Archivos CSV de consumos.
- Archivos JSON de materiales.
- Datos provenientes de Azure SQL Database.

La función principal de esta carpeta es actuar como zona de aterrizaje (*landing zone*) donde los archivos ingresan sin procesamiento previo.

5.6 Pipeline de Azure Data Factory

Una vez depositados los archivos en la carpeta input, Azure Data Factory ejecuta automáticamente pipelines de ingesta previamente configurados.

Estos pipelines realizan:

1. Lectura de los archivos de entrada.
2. Validación estructural básica.
3. Conversión de formatos según la fuente.
4. Copia hacia la capa RAW.
5. Registro de ejecución y monitoreo.

El objetivo de esta etapa es estandarizar la forma en que los datos son almacenados antes de comenzar cualquier transformación analítica.

Nombre de actividad	Estado de...	Tipo de...	Inicio de la ejecu...	Duraci...	Entorno de ejecución de integración	Id.
Region	Correcto	Copy data	5/5/2024, 7:12:38 PM	1h	AutoResolveIntegrationRuntime (Shared South)	
RegionParquet	Correcto	Copy data	5/5/2024, 7:13:38 PM	1h	AutoResolveIntegrationRuntime (Shared South)	
Region	Correcto	Copy data	5/5/2024, 7:14:38 PM	1h	AutoResolveIntegrationRuntime (Shared South)	

5.7 Automatización del Pipeline de Ingesta

Con el objetivo de garantizar una correcta secuencia de ejecución dentro de la arquitectura de datos, se configuró un trigger automático en Azure Data Factory para el pipeline de ingesta.

Este trigger fue programado diariamente a las:

09:30 AM

La lógica detrás de esta configuración responde a una necesidad operativa típica en entornos reales de construcción, donde los consumos de materiales generados durante la jornada laboral del día anterior deben encontrarse disponibles antes de iniciar los procesos analíticos y de transformación.

De esta manera, el pipeline de ingesta ejecuta previamente:

- La recepción de nuevos archivos.
- La copia hacia la capa RAW.
- La conversión de formatos.
- La actualización de datasets base.

El objetivo es asegurar que toda la información más reciente se encuentre correctamente almacenada y disponible dentro del Data Lake antes de comenzar el procesamiento en Azure Databricks.

Nombre *

TFI_IngestaRAW

Descripción

Tipo *

Programar

Fecha de inicio * 

5/8/2026 9:30:00 AM

Zona horaria * 

Buenos Aires (UTC-3)

Periodicidad * 

Cada 1

Días

 Opciones avanzadas de periodicidad

☐ Especificar una fecha de finalización

Asociaciones

Nuevo

iniciar desencadenador 

☒ iniciar desencadenador al crear

Aceptar

Cancelar

Posteriormente, se configuró un segundo trigger programado para las:

10:00 AM

Este segundo trigger ejecuta los notebooks de Databricks encargados de:

- Procesamiento de la capa Bronze.
- Validaciones en Silver.
- Generación de tablas Gold.
- Escritura en Delta Lake.
- Optimización de tablas analíticas.

La diferencia horaria entre ambos triggers fue definida estratégicamente para garantizar que el proceso de ingesta finalice correctamente antes de iniciar las transformaciones.

5.8 Capa RAW

Luego del proceso de ingesta, los datos son almacenados nuevamente dentro del mismo contenedor del Blob Storage, pero ahora en una carpeta denominada:

/raw

La carpeta RAW representa la primera capa oficial del Data Lake y constituye la fuente base de lectura para el procesamiento posterior realizado en Azure Databricks.

En esta capa los datos se almacenan prácticamente sin transformaciones de negocio, preservando la estructura original de cada fuente.

Esto permite:

- Mantener trazabilidad.
- Recuperar información original ante errores.
- Reprocesar datos cuando sea necesario.
- Separar claramente ingestión de transformación.

5.9 Conversión y Estandarización de Archivos

Durante la ingesta se realizó una estandarización específica para cada tipo de fuente de datos.

5.9.1 Conversión de archivos CSV

Los archivos CSV originales fueron ingeridos mediante Azure Data Factory utilizando datasets de tipo: DelimitedText (CSV Text)

Esta configuración permitió:

- Interpretar correctamente delimitadores.
- Definir encabezados.
- Controlar esquemas.
- Estandarizar codificación y formatos.

Los archivos fueron almacenados en RAW manteniendo su naturaleza tabular.

5.9.2 Conversión de archivos JSON

El catálogo de materiales fue cargado utilizando datasets de tipo:

JSON

La utilización de JSON permitió trabajar con datos semiestructurados simulando un escenario similar al consumo de APIs REST.

Azure Data Factory realizó:

- Parsing automático del JSON.
- Validación de estructura.
- Lectura jerárquica de objetos.
- Persistencia organizada en la capa RAW.

Esto facilitó posteriormente la lectura dinámica desde Spark en Databricks.

5.9.3 Conversión desde Azure SQL Database a Parquet

La información de proveedores almacenada en Azure SQL Database fue ingerida y exportada hacia formato:

Parquet

La elección de Parquet se realizó debido a sus ventajas en escenarios analíticos:

- Formato columnar.
- Mayor compresión.
- Lectura optimizada.
- Mejor rendimiento en Spark.
- Menor consumo de almacenamiento.

Azure Data Factory realizó la extracción desde SQL y la persistencia automática en archivos Parquet dentro de la carpeta RAW.

5.10 Relación con Azure Databricks

La carpeta RAW fue utilizada posteriormente como fuente principal de lectura dentro de Azure Databricks para construir las capas:

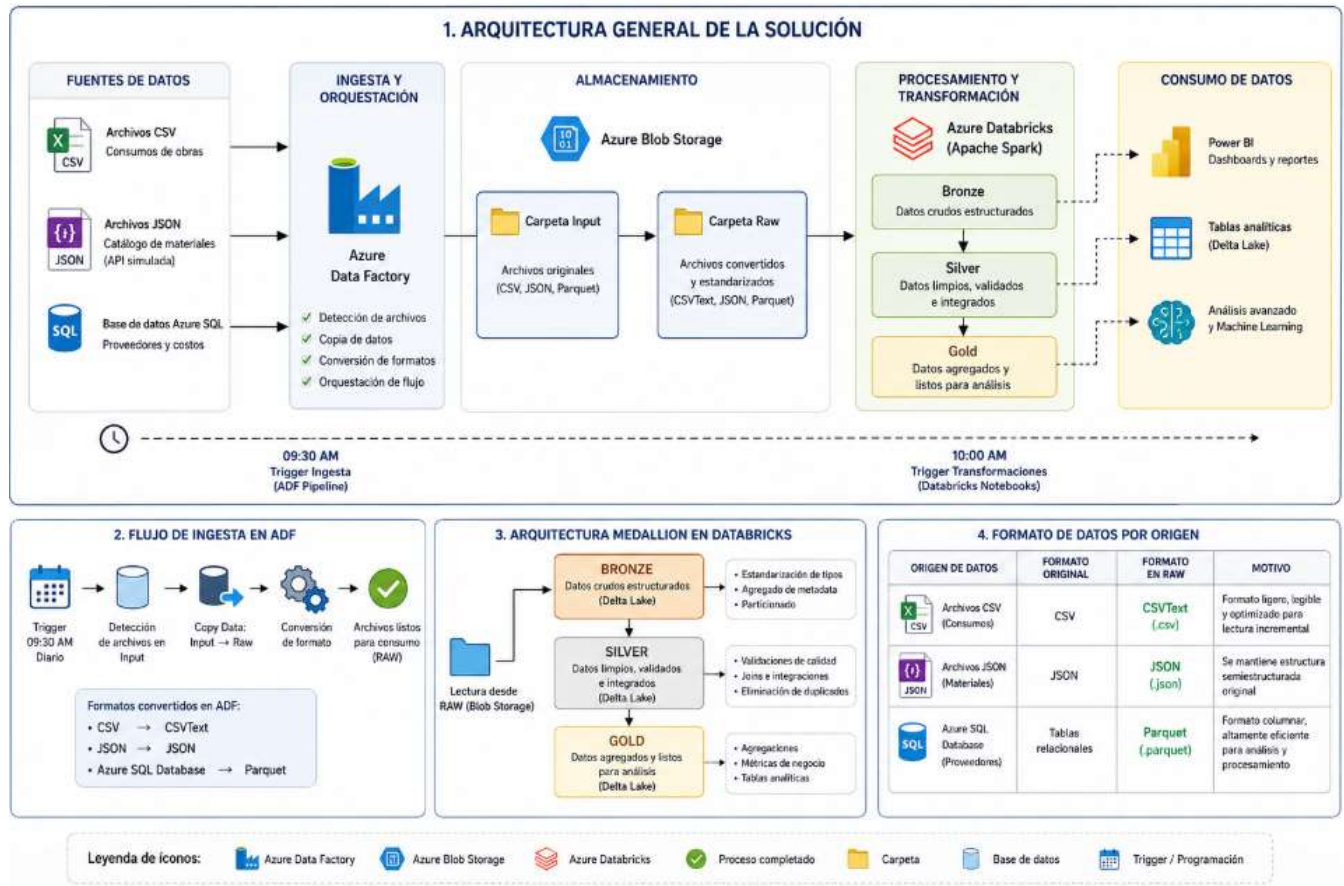
Bronze → Silver → Gold

Databricks consume directamente los datos almacenados en RAW para:

- Limpieza.
- Validaciones.
- Transformaciones.
- Enriquecimiento.
- Escritura en formato Delta Lake.

De esta manera, la arquitectura implementada mantiene una clara separación entre:

Etapa	Responsabilidad
Azure Data Factory	Ingesta y movimiento de datos
Azure Blob Storage	Persistencia centralizada
Azure Databricks	Procesamiento y transformación



6 Introducción a la Arquitectura Medallion

Con el objetivo de construir una solución escalable, organizada y alineada con las buenas prácticas modernas de ingeniería de datos, en el presente trabajo se implementó una arquitectura tipo Medallion utilizando Azure Databricks y Delta Lake.

La arquitectura Medallion propone dividir el ciclo de vida de los datos en múltiples capas lógicas, donde cada una cumple una función específica dentro del procesamiento y refinamiento de la información.

Este enfoque permite mejorar:

- La calidad de los datos.
- La trazabilidad.
- La escalabilidad del pipeline.
- El rendimiento analítico.
- La gobernanza de la información.

La arquitectura implementada se compone de tres capas principales:

6.1.1 Capa Bronze

La capa Bronze representa el primer nivel de procesamiento dentro de Databricks y actúa como punto de aterrizaje de los datos provenientes de la capa RAW almacenada en Azure Blob Storage.

En esta etapa los datos son incorporados prácticamente en su estado original, aplicando únicamente transformaciones mínimas orientadas a:

- Estandarización básica.
- Conversión a formato Delta.
- Agregado de metadata técnica.
- Preservación de trazabilidad.

El objetivo principal de esta capa es conservar una copia estructurada y reutilizable de los datos ingeridos.

6.1.2 Capa Silver

La capa Silver corresponde al nivel donde se aplican las principales reglas de negocio y calidad de datos.

En esta etapa se realizan procesos como:

- Limpieza de registros.
- Validaciones de integridad.
- Eliminación de duplicados.
- Estandarización de formatos.
- Integración entre múltiples fuentes.
- Joins y enriquecimiento de información.

El resultado son datasets limpios, consistentes y preparados para el análisis analítico posterior.

6.1.3 Capa Gold

La capa Gold representa el nivel final de consumo analítico.

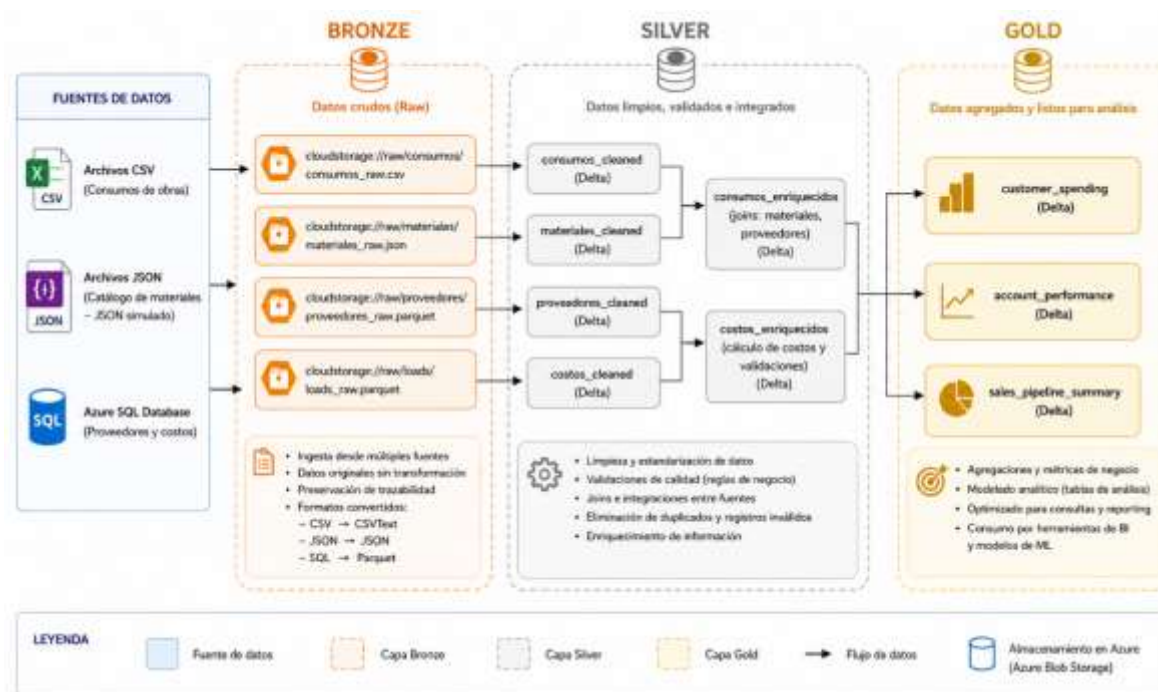
Aquí se generan tablas optimizadas orientadas al negocio, incluyendo:

- Métricas.
- KPIs.
- Agregaciones.
- Indicadores gerenciales.
- Tablas analíticas para reporting y visualización.

Estas tablas permiten transformar grandes volúmenes de datos operativos en información útil para la toma de decisiones.

La implementación de esta arquitectura permitió estructurar el pipeline de forma clara y modular, facilitando tanto el mantenimiento como la futura escalabilidad de la solución.

Además, el uso conjunto de Azure Databricks y Delta Lake permitió aprovechar capacidades avanzadas de procesamiento distribuido, versionado y optimización de datos.



7 Capa Bronze

7.1 Introducción

La capa Bronze representa el primer nivel de procesamiento dentro de la arquitectura Medallion implementada en Azure Databricks. Su principal objetivo es recibir los datos provenientes de la capa RAW y almacenarlos en un formato estructurado y optimizado para posteriores transformaciones.

En esta etapa los datos son incorporados prácticamente en su estado original, preservando la mayor cantidad posible de información proveniente de las fuentes de origen. La finalidad principal de la capa Bronze es garantizar trazabilidad, historización y reprocesamiento de los datos.

Dentro del presente proyecto, la capa Bronze actúa como punto de entrada al entorno analítico de Databricks, permitiendo desacoplar completamente los procesos de ingesta realizados en Azure Data Factory de las transformaciones posteriores de negocio.

7.2 Objetivos de la Capa Bronze

Los principales objetivos de esta capa son:

- Centralizar los datos ingeridos desde múltiples fuentes.
- Preservar los datos originales.
- Estandarizar estructuras básicas.
- Convertir datasets a formato Delta Lake.
- Generar una base reutilizable para las capas superiores.
- Mantener trazabilidad técnica de los archivos procesados.

7.3 Origen de los Datos

La capa Bronze consume directamente la información almacenada previamente en la carpeta RAW del Azure Blob Storage.

Las fuentes utilizadas fueron:

Fuente	Formato en RAW
Consumos de materiales	CSVText
Catálogo de materiales	JSON
Proveedores	Parquet

Todos los datasets fueron leídos desde Azure Databricks utilizando PySpark.

7.4 Lectura de Archivos en Databricks

Dentro de los notebooks de Databricks se realizó la lectura de cada fuente utilizando Spark DataFrames.

La lectura se efectuó directamente desde Azure Blob Storage mediante rutas WASBS, permitiendo acceso distribuido y escalable a los archivos almacenados.

Lectura de consumos (CSV)

Se configuró Spark para interpretar:

- Encabezados.
- Delimitadores.
- Inferencia de esquemas.
- Tipos de datos.

El objetivo fue cargar correctamente el dataset masivo de consumos generado previamente.

```

▶ 08:15 (1 m) 9

# Leer ventas y guardar en Delta particionado por fecha
consumo_materiales_bronze = spark.read.format("csv") \
    .option("header", True) \
    .load(f"{BASE}/raw/consumos_materiales_500000.txt")

consumo_materiales_bronze.write.format("delta") \
    .mode("overwrite") \
    .partitionBy("fecha") \
    .save(f"{BASE}/bronze/consumo_materiales")

print("consumo_materiales guardado:", consumo_materiales_bronze.count(), "filas")

▶ (9) trabajos de Spark

> consumo_materiales_bronze: pyspark.sql.connect.dataframe.DataFrame = [consumo_id: string, material_id: string ... 13 campos adicionales]
consumo_materiales guardado: 500000 filas

```

7.5 Lectura de materiales (JSON)

El archivo JSON fue leído utilizando capacidades nativas de Spark para procesamiento de datos semiestructurados.

Esto permitió:

- Inferir estructuras jerárquicas.
- Interpretar objetos JSON.
- Mantener flexibilidad en el esquema.

```

from pyspark.sql.functions import explode, col

materiales_bronze = spark.read.format("json") \
    .option("multiline", True) \
    .load(f"{BASE}/raw/materiales_200.json")

materiales_bronze = materiales_bronze \
    .select(explode(col("materiales")).alias("m")) \
    .select("m.*")

materiales_bronze.write.format("delta") \
    .mode("overwrite") \
    .save(f"{BASE}/bronze/materiales")

print("materiales guardado:", materiales_bronze.count(), "filas")

```

► (4) trabajos de Spark

► materiales_bronze: pyspark.sql.connect.dataframe.DataFrame = [categoria: string, fecha_actualizacion: string ... 8 campos adicionales]

materiales guardado: 200 filas

7.6 Lectura de proveedores (Parquet)

La información proveniente de Azure SQL Database ya había sido convertida previamente a formato Parquet durante el proceso de ingesta.

Spark realizó la lectura de forma optimizada aprovechando las ventajas del formato columnar.

```

08:15 (2 s) 13

proveedores_bronze = spark.read.format("parquet") \
    .load(f"{BASE}/raw/dbo.proveedores.parquet")

proveedores_bronze.write.format("delta") \
    .mode("overwrite") \
    .save(f"{BASE}/bronze/proveedores")

print("proveedores guardado:", proveedores_bronze.count(), "filas")

```

► (4) trabajos de Spark

► proveedores_bronze: pyspark.sql.connect.dataframe.DataFrame = [proveedor_id: integer, nombre: string ... 13 campos adicionales]

proveedores guardado: 51 filas

7.7 Conversión a Delta Lake

Uno de los procesos más importantes de la capa Bronze fue la conversión de los datasets hacia formato Delta Lake.

Esta decisión permitió incorporar funcionalidades avanzadas como:

- Versionado.
- ACID Transactions.
- Time Travel.

- Optimización de consultas.
- Mayor rendimiento en procesamiento distribuido.

7.7.1 Escritura de Datos en Bronze

Una vez procesados, los datasets fueron almacenados nuevamente en Azure Blob Storage dentro de rutas específicas para la capa Bronze.

Ejemplo conceptual:

```
/bronze/consumos
  /bronze/materiales
  /bronze/proveedores
```

Cada dataset fue persistido en formato Delta.

```
▶ 08:15 (3 s)

print("=== consumo_materiales ===")
spark.read.format("delta") \
  .load(f"{BASE}/bronze/consumo_materiales") \
  .printSchema()

spark.read.format("delta") \
  .load(f"{BASE}/bronze/consumo_materiales") \
  .show(5)

print("=== materiales ===")
spark.read.format("delta") \
  .load(f"{BASE}/bronze/materiales") \
  .printSchema()

spark.read.format("delta") \
  .load(f"{BASE}/bronze/materiales") \
  .show(5)

print("=== proveedores ===")
spark.read.format("delta") \
  .load(f"{BASE}/bronze/proveedores") \
  .printSchema()

spark.read.format("delta") \
  .load(f"{BASE}/bronze/proveedores") \
  .show(5)

▶ (4) trabajos de Spark
```

7.7.2 Beneficios de la Capa Bronze

La implementación de esta capa aporta múltiples ventajas:

Beneficios Técnicos

- Persistencia optimizada.
- Escalabilidad.
- Trazabilidad.
- Reprocesamiento sencillo.
- Compatibilidad con Spark.

7.7.3 Beneficios Operativos

- Separación clara entre ingesta y transformación.
- Mayor mantenibilidad.
- Estandarización inicial de datos.
- Base sólida para validaciones posteriores.

7.8 Relación con las Capas Superiores

La capa Bronze constituye la fuente principal de lectura para la construcción de la capa Silver.

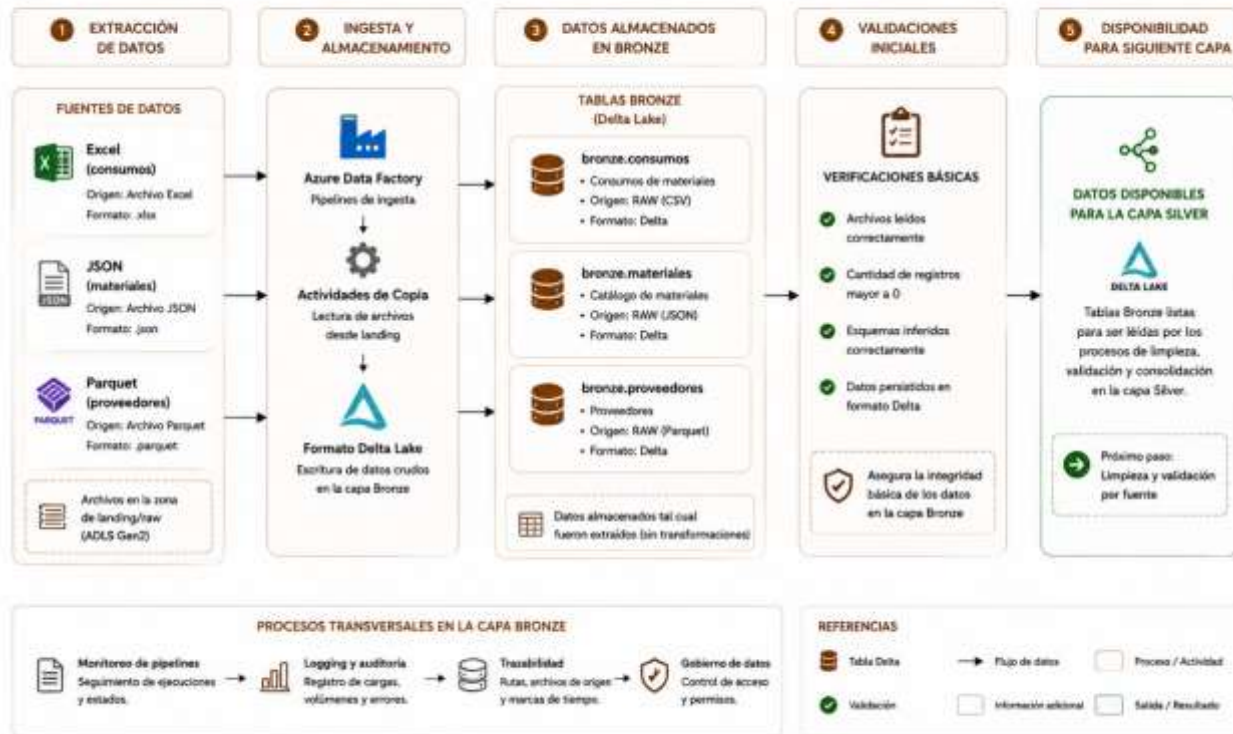
A partir de esta capa comienzan los procesos de:

- Limpieza avanzada.
- Validaciones de calidad.
- Integraciones entre datasets.
- Reglas de negocio.
- Generación de información analítica.

De esta manera, Bronze funciona como el punto de transición entre los datos crudos provenientes del Data Lake y el procesamiento analítico realizado en Databricks.

FLUJO DE TABLAS – CAPA BRONZE

Ingesta de datos crudos desde múltiples orígenes y almacenamiento en formato Delta Lake



8 Capa Silver

8.1 Introducción

La capa Silver representa el núcleo principal de procesamiento dentro de la arquitectura Medallion implementada en el proyecto. En esta etapa comienzan a aplicarse las transformaciones de negocio, validaciones de calidad y procesos de integración necesarios para convertir los datos crudos provenientes de Bronze en datasets confiables y preparados para análisis posteriores.

El notebook desarrollado en Azure Databricks para esta capa fue construido utilizando PySpark y permitió trabajar de forma distribuida sobre grandes volúmenes de datos provenientes de múltiples fuentes.

8.2 Objetivo de la Capa Silver

El objetivo principal de esta capa fue transformar los datos provenientes de Bronze en datasets limpios, consistentes y listos para consumo analítico.

Para ello se implementaron procesos orientados a:

- Validación de calidad de datos.
- Integración entre múltiples fuentes.
- Limpieza de registros inválidos.
- Eliminación de duplicados.
- Generación de nuevas columnas analíticas.
- Estandarización de estructuras.

La capa Silver funciona como punto intermedio entre los datos operacionales y las tablas analíticas finales de negocio.

8.3 Lectura de Datos desde Bronze

El proceso comenzó con la lectura de las tablas Delta generadas previamente en la capa Bronze.

Los datasets consumidos fueron:

Dataset	Origen
Consumos de materiales	Bronze
Catálogo de materiales	Bronze
Proveedores	Bronze

Cada tabla fue cargada como Spark DataFrame dentro de Databricks utilizando PySpark.

Esto permitió realizar transformaciones distribuidas y escalables sobre grandes volúmenes de información.

```

▶ 08:12 (3 s) 6

# Leer las cuatro tablas desde Bronze
consumo_materiales = spark.read.format("delta").load(f"{BASE}/bronze/consumo_materiales")
materiales = spark.read.format("delta").load(f"{BASE}/bronze/materiales")
proveedores = spark.read.format("delta").load(f"{BASE}/bronze/proveedores")

print("Filas en consumo_materiales (raw):", consumo_materiales.count())
print("Filas en materiales (raw):", materiales.count())
print("Filas en proveedores (raw):", proveedores.count())

▶ (8) trabajos de Spark

> consumo_materiales: pyspark.sql.connect.dataframe.DataFrame = [consumo_id: string, material_id: string ... 13 campos adicionales]
> materiales: pyspark.sql.connect.dataframe.DataFrame = [categoria: string, fecha_actualizacion: string ... 8 campos adicionales]
> proveedores: pyspark.sql.connect.dataframe.DataFrame = [proveedor_id: integer, nombre: string ... 13 campos adicionales]

Filas en consumo_materiales (raw): 500000
Filas en materiales (raw): 200
Filas en proveedores (raw): 51

```

8.4 Transformaciones Aplicadas

Uno de los puntos centrales de la capa Silver fue la aplicación de múltiples transformaciones sobre los datasets, cumpliendo con los requerimientos definidos en la consigna del trabajo integrador.

8.4.1 1. Integración de Datos mediante JOINS

La primera transformación importante implementada fue la integración de datasets mediante operaciones JOIN entre distintas fuentes de datos.

Esto permitió relacionar:

- Consumos de materiales.
- Información descriptiva de materiales.
- Datos de proveedores.

Gracias a estas integraciones fue posible enriquecer cada registro de consumo incorporando información adicional como:

- Nombre del material.
- Categoría.
- Precio unitario.
- Proveedor asociado.
- Datos descriptivos complementarios.

Esta transformación permitió consolidar información dispersa en un único dataset integrado y consistente.

▶ ▼ ✓ 08:12 (11 s)

```
c = consumo_materiales_validos.alias("c")
m = materiales_validos.alias("m")

validacion_materiales = c.join(
    m,
    col("c.material_id") == col("m.material_id"),
    "left"
)

consumos_material_inexistente = validacion_materiales.filter(
    col("m.material_id").isNull()
).select(
    col("c.*"),
    lit("Material inexistente en catálogo").alias("motivo_error")
)

consumos_silver_validos = validacion_materiales.filter(
    col("m.material_id").isNotNull()
).select(
    col("c.consumo_id"),
    col("c.material_id"),
    col("c.fecha"),
    col("c.obra"),
    col("c.etapabilidad"),
    col("c.responsable"),
    col("c.unidad").alias("unidad_consumo"),
    col("c.cantidad"),
    col("m.material").alias("nombre_material"),
    col("m.categoria").alias("categoria_material"),
    col("m.precio_unitario_estimado"),
    col("m.proveedor").alias("proveedor_material")
)
```

8.4.2 2. Limpieza y Filtrado de Registros

Otra de las transformaciones aplicadas correspondió a procesos de limpieza y filtrado de registros inválidos.

Durante esta etapa se detectaron inconsistencias presentes en los datasets originales provenientes de distintas fuentes.

Entre las principales tareas realizadas se incluyeron:

- Eliminación de registros incompletos.
- Filtrado de datos inconsistentes.
- Revisión de campos obligatorios.
- Corrección de estructuras incompatibles.

El objetivo principal fue garantizar que únicamente registros válidos continúen hacia las etapas analíticas posteriores.

8.4.3 3. Eliminación de Duplicados

La capa Silver también incorporó procesos de deduplicación sobre los datasets procesados.

Se eliminaron registros duplicados utilizando claves específicas de cada tabla.

Esto permitió evitar:

- Sobreconteo de consumos.
- Duplicación de costos.
- Inconsistencias analíticas.
- Distorsión de métricas finales.

La eliminación de duplicados resultó fundamental para garantizar confiabilidad en las tablas Gold posteriores.

```

08:12 (6 s) 18
consumo_materiales_dedup = consumo_tipado.dropDuplicates(["consumo_id"])

print("Filas originales:", consumo_tipado.count())
print("Filas sin duplicados:", consumo_materiales_dedup.count())

materiales_dedup = materiales_limpios.dropDuplicates(["material_id"])

print("Filas originales:", materiales_limpios.count())
print("Filas sin duplicados:", materiales_dedup.count())

proveedores_dedup = proveedores_limpios.dropDuplicates(["proveedor_id"])

print("Filas originales:", proveedores_limpios.count())
print("Filas sin duplicados:", proveedores_dedup.count())

(17) trabajos de Spark
> consumo_materiales_dedup: pyspark.sql.connect.dataframe.DataFrame = [consumo_id: string, material_id: string ... 13 campos adicionales]
> materiales_dedup: pyspark.sql.connect.dataframe.DataFrame = [categoria: string, fecha_actualizacion: string ... 8 campos adicionales]
> proveedores_dedup: pyspark.sql.connect.dataframe.DataFrame = [proveedor_id: integer, nombre: string ... 13 campos adicionales]

```

8.4.4 4. Generación de Nuevas Columnas

Dentro del notebook también se realizaron cálculos de nuevas columnas derivadas orientadas al análisis económico y operativo de las obras.

Entre las principales columnas generadas se incluyeron:

- Costo total por consumo.
- Indicadores auxiliares.
- Métricas derivadas de cantidad y precio unitario.

Estas transformaciones permitieron enriquecer analíticamente los datasets antes de su utilización en la capa Gold.

8.4.5 Validaciones de Calidad de Datos

Uno de los aspectos más importantes implementados en la capa Silver fue el control de calidad de datos.

Se definieron múltiples reglas de validación orientadas a detectar inconsistencias dentro de la información procesada.

8.4.5.1 Validación 1 – Campos Obligatorios

Se verificó la existencia de valores nulos en campos críticos como:

- consumo_id
- material_id
- proveedor_id
- cantidad

Los registros que no cumplían esta condición fueron considerados inválidos.

```

08:12 (7 s) 20

from pyspark.sql.functions import col

consumo_materiales_nulos = consumo_materiales_dedup.filter(
    col("consumo_id").isNull() |
    col("material_id").isNull() |
    col("fecha").isNull() |
    col("cantidad").isNull()
)

consumo_materiales_validos = consumo_materiales_dedup.filter(
    col("consumo_id").isNotNull() &
    col("material_id").isNotNull() &
    col("fecha").isNotNull() &
    col("cantidad").isNotNull()
)

print("Consumos válidos:", consumo_materiales_validos.count())
print("Consumos con nulos:", consumo_materiales_nulos.count())

(6) trabajos de Spark
> consumo_materiales_nulos: pyspark.sql.connect.dataframe.DataFrame = [consumo_id: string, material_id: string ... 13 campos adicionales]
> consumo_materiales_validos: pyspark.sql.connect.dataframe.DataFrame = [consumo_id: string, material_id: string ... 13 campos adicionales]

Consumos válidos: 500000
Consumos con nulos: 0

```

8.4.5.2 Validación 2 – Cantidades Mayores a Cero

Se implementó una validación para garantizar que las cantidades consumidas sean mayores a cero.

Esto permitió detectar:

- Errores de carga.
- Registros inconsistentes.
- Valores inválidos.

```

08:12 (7 s) 27

consumos_cantidad_invalida = consumo_materiales_validos.filter(
    col("cantidad") <= 0
)

consumo_materiales_validos = consumo_materiales_validos.filter(
    col("cantidad") > 0
)

print("Consumos con cantidad inválida:", consumos_cantidad_invalida.count())
print("Consumos válidos finales:", consumo_materiales_validos.count())

(6) trabajos de Spark
> consumo_materiales_validos: pyspark.sql.connect.dataframe.DataFrame = [consumo_id: string, material_id: string ... 13 campos adicionales]
> consumos_cantidad_invalida: pyspark.sql.connect.dataframe.DataFrame = [consumo_id: string, material_id: string ... 13 campos adicionales]

Consumos con cantidad inválida: 0
Consumos válidos finales: 500000

```

8.4.5.3 Validación 3 – Integridad entre Datasets

También se verificó la consistencia entre datasets relacionados mediante validaciones de integridad referencial.

Por ejemplo:

- materiales inexistentes en consumos,
- proveedores sin relación válida,
- IDs sin correspondencia.

```

▶ 08:12 (1 s) 24

proveedores_nulos = proveedores_dedup.filter(
    col("proveedor_id").isNull() |
    col("nombre").isNull() |
    col("cuit").isNull()
)

proveedores_validos = proveedores_dedup.filter(
    col("proveedor_id").isNotNull() &
    col("nombre").isNotNull() &
    col("cuit").isNotNull()
)

print("Proveedores válidos:", proveedores_validos.count())
print("Proveedores con nulos:", proveedores_nulos.count())

▶ (6) trabajos de Spark

> proveedores_nulos: pyspark.sql.connect.dataframe.DataFrame = [proveedor_id: integer, nombre: string ... 13 campos adicionales]
> proveedores_validos: pyspark.sql.connect.dataframe.DataFrame = [proveedor_id: integer, nombre: string ... 13 campos adicionales]

Proveedores válidos: 51
Proveedores con nulos: 0

```

8.4.6 Manejo de Registros Inválidos

El pipeline fue diseñado para que los errores de calidad no detuvieran completamente el procesamiento.

En lugar de interrumpir la ejecución:

- Los registros inválidos fueron separados del flujo principal.
- Los datos válidos continuaron su procesamiento.
- Los errores pudieron ser identificados posteriormente para auditoría y revisión.

Esta estrategia permitió mantener estabilidad operativa dentro del pipeline.

```

08:12 (5 s) 33
from pyspark.sql.functions import lit

# Errores de consumos por nulos
errores_consumos_nulos = consumo_materiales_nulos.withColumn(
    "motivo_error", lit("Campos obligatorios nulos")
)

# Errores de materiales por nulos
errores_materiales_nulos = materiales_nulos.withColumn(
    "motivo_error", lit("Campos obligatorios nulos")
)

# Errores de proveedores por nulos
errores_proveedores_nulos = proveedores_nulos.withColumn(
    "motivo_error", lit("Campos obligatorios nulos")
)

print("Errores en consumos:", errores_consumos_nulos.count())
print("Errores en materiales:", errores_materiales_nulos.count())
print("Errores en proveedores:", errores_proveedores_nulos.count())

```

► (9) trabajos de Spark

```

> errores_consumos_nulos: pyspark.sql.connect.dataframe.DataFrame = [consumo_id: string, material_id: string ... 14 campos adicionales]
> errores_materiales_nulos: pyspark.sql.connect.dataframe.DataFrame = [categoria: string, fecha_actualizacion: string ... 9 campos adicionales]
> errores_proveedores_nulos: pyspark.sql.connect.dataframe.DataFrame = [proveedor_id: integer, nombre: string ... 14 campos adicionales]

```

Errores en consumos: 0
 Errores en materiales: 0
 Errores en proveedores: 0

8.5 Persistencia de Datos en Silver

Una vez finalizados los procesos de transformación y validación, los datasets resultantes fueron almacenados nuevamente en Azure Blob Storage dentro de la capa Silver utilizando formato Delta Lake.

Ejemplo conceptual:

```

/silver/consumos_materiales
  /silver/materiales_limpios
  /silver/proveedores_limpios

```

La utilización de Delta Lake permitió:

- Versionado de datos.
- Mejor rendimiento de lectura.
- Compatibilidad con Spark.
- Escalabilidad analítica.

```

08:12 (11 s) 35

from pyspark.sql.functions import col, lit

c = consumo_materiales_validos.alias("c")
m = materiales_validos.alias("m")

validacion_materiales = c.join(
    m,
    on="material_id",
    how="left"
)

consumos_material_inexistente = validacion_materiales.filter(
    col("m.material").isNull()
).withColumn(
    "motivo_error", lit("Material inexistente en catálogo")
)

consumos_silver_validos = validacion_materiales.filter(
    col("m.material").isNotNull()
)

print("Consumos válidos antes de integridad:", consumo_materiales_validos.count())
print("Consumos con material inexistente:", consumos_material_inexistente.count())
print("Consumos válidos finales:", consumos_silver_validos.count())

(13) trabajos de Spark

> c: pyspark.sql.connect.dataframe.DataFrame = [consumo_id: string, material_id: string ... 13 campos adicionales]
> consumos_material_inexistente: pyspark.sql.connect.dataframe.DataFrame = [material_id: string, consumo_id: string ... 23 campos adicionales]
> consumos_silver_validos: pyspark.sql.connect.dataframe.DataFrame = [material_id: string, consumo_id: string ... 22 campos adicionales]
> m: pyspark.sql.connect.dataframe.DataFrame = [categoria: string, fecha_actualizacion: string ... 8 campos adicionales]
> validacion_materiales: pyspark.sql.connect.dataframe.DataFrame = [material_id: string, consumo_id: string ... 22 campos adicionales]

Consumos válidos antes de integridad: 500000
Consumos con material inexistente: 0
Consumos válidos finales: 500000

```

8.6 Importancia de la Capa Silver

La capa Silver representa el punto donde los datos comienzan a transformarse en información confiable para análisis de negocio.

Gracias a las validaciones y transformaciones implementadas fue posible:

- Mejorar la calidad de los datos.
- Garantizar consistencia.
- Integrar múltiples fuentes.
- Preparar datasets analíticos.
- Reducir errores en métricas finales.

Esta capa se convirtió así en la base principal para la construcción de tablas analíticas y agregaciones utilizadas posteriormente en la capa Gold.

Tabla Integrada Generada para la Capa Gold

Como resultado del proceso de integración y validación, se generó una tabla consolidada dentro de la capa Silver que posteriormente fue utilizada como fuente principal de lectura para la construcción de las tablas analíticas Gold.

Esta tabla integrada contiene información unificada de:

- Consumos.
- Materiales.
- Proveedores.
- Costos.
- Obras.
- Etapas constructivas.

La estructura consolidada permitió simplificar significativamente los procesos analíticos posteriores, evitando realizar nuevamente integraciones complejas en la capa Gold.

Ejemplo conceptual de la tabla resultante:

<i>Campo</i>	<i>Descripción</i>
<i>consumo_id</i>	Identificador del consumo
<i>material_id</i>	Material utilizado
<i>nombre_material</i>	Descripción del material
<i>categoria</i>	Categoría del producto
<i>proveedor</i>	Proveedor asociado
<i>obra</i>	Obra correspondiente
<i>etapa</i>	Etapas constructivas
<i>cantidad</i>	Cantidad consumida
<i>costo_unitario</i>	Valor unitario
<i>costo_total</i>	Costo calculado

Esta estrategia permitió que la capa Gold pudiera enfocarse exclusivamente en procesos analíticos y agregaciones de negocio, utilizando una fuente de datos ya limpia, integrada y validada.

8.7 Optimización de Tablas Delta con OPTIMIZE

Una vez generadas las tablas Delta dentro de la capa Silver, se aplicaron procesos de optimización utilizando funcionalidades nativas de Delta Lake.

Para ello se utilizó el comando:

8.7.1 Optimize

El objetivo principal de esta optimización fue mejorar el rendimiento de lectura y consulta sobre las tablas procesadas.

Durante el procesamiento distribuido en Spark, los datos suelen almacenarse en múltiples archivos pequeños debido a la paralelización de tareas. Esto puede afectar negativamente el rendimiento analítico.

La ejecución de OPTIMIZE permitió:

- Compactar archivos pequeños.
- Mejorar tiempos de lectura.
- Reducir operaciones de I/O.
- Optimizar consultas posteriores.
- Incrementar eficiencia en la capa Gold.

Esta optimización resulta especialmente importante en escenarios de grandes volúmenes de datos y procesamiento analítico intensivo.

Además, permitió preparar las tablas para futuras consultas desde herramientas de visualización y reporting.

8.7.2 Uso de DESCRIBE HISTORY

Otra funcionalidad importante implementada sobre las tablas Delta fue el uso del comando:

DESCRIBE HISTORY

Esta característica permitió acceder al historial completo de operaciones realizadas sobre cada tabla Delta.

Gracias a esta funcionalidad fue posible visualizar:

- Fecha y hora de modificaciones.
- Operaciones ejecutadas.
- Versiones generadas.
- Usuarios o procesos responsables.
- Cantidad de archivos modificados.
- Métricas de escritura y optimización.

La utilización de DESCRIBE HISTORY aportó importantes capacidades de:

- Auditoría.
- Trazabilidad.
- Gobierno de datos.
- Seguimiento operativo.

Además, permitió demostrar una de las principales ventajas de Delta Lake frente a formatos tradicionales de almacenamiento, incorporando capacidades avanzadas de versionado y control histórico de datos.

Estas funcionalidades fortalecieron la robustez técnica del pipeline implementado y aportaron características alineadas con arquitecturas modernas de ingeniería de datos empresariales.



FLUJO DE TABLAS – CAPA SILVER

Integración, limpieza, validación y construcción de tabla consolidada para consumo en Gold



9 Capa Gold

9.1 Introducción

La capa Gold representa el nivel final dentro de la arquitectura Medallion implementada en el proyecto. Esta capa tiene como objetivo transformar los datasets procesados y validados en Silver en información analítica orientada al negocio.

A diferencia de las capas anteriores, donde el foco estuvo puesto en la ingestión, limpieza y consolidación de datos, la capa Gold se centra en la generación de métricas, agregaciones e indicadores útiles para la toma de decisiones.

El notebook desarrollado en Azure Databricks para esta etapa fue construido utilizando PySpark y tuvo como finalidad crear tablas analíticas optimizadas a partir de la tabla integrada generada previamente en Silver:

`/silver/consumos_materiales_proveedores`

Esta tabla consolidada actuó como fuente principal de lectura para la generación de datasets analíticos finales.

9.2 Objetivo de la Capa Gold

El objetivo principal de esta capa fue construir tablas de negocio listas para consumo analítico y visualización.

Entre los principales propósitos de la capa Gold se encuentran:

- Generación de KPIs.
- Construcción de métricas de negocio.
- Agregaciones analíticas.
- Optimización para reporting.
- Preparación para herramientas BI.
- Simplificación de consultas gerenciales.

La capa Gold representa el nivel de mayor valor dentro del pipeline de datos, ya que transforma información técnica y operacional en conocimiento útil para el análisis empresarial.

9.3 Lectura de Datos desde Silver

El notebook de Gold comenzó leyendo la tabla integrada construida en la capa Silver.

La tabla utilizada fue:

`/silver/consumos_materiales_proveedores`

Esta tabla ya contenía:

- Datos limpios.
- Registros validados.
- Información integrada mediante JOINS.
- Costos calculados.
- Relación entre consumos, materiales y proveedores.

Gracias a esto, la capa Gold pudo enfocarse exclusivamente en procesos analíticos sin necesidad de repetir tareas de limpieza o integración.

```

▶ 09:37 (1 s) 6

from pyspark.sql.functions import sum, col, round

# Leer la tabla Silver de consumos integrados
silver_final = spark.read.format("delta").load(
    f"{BASE}/silver/consumos_materiales_proveedores"
)

print("Filas en Silver:", silver_final.count())

silver_final.printSchema()
▶ (2) trabajos de Spark

> silver_final: pyspark.sql.connect.dataframe.DataFrame = [consumo_id: string, material_id: string ... 22 campos adicionales]

Filas en Silver: 500000
root
 |-- consumo_id: string (nullable = true)
 |-- material_id: string (nullable = true)
 |-- fecha: date (nullable = true)
 |-- obra: string (nullable = true)
 |-- etapabilidad: string (nullable = true)
 |-- responsable: string (nullable = true)
 |-- unidad_consumo: string (nullable = true)
 |-- cantidad: double (nullable = true)
 |-- nombre_material: string (nullable = true)
 |-- categoria_material: string (nullable = true)
 |-- precio_unitario_estimado: double (nullable = true)
 |-- proveedor_material: string (nullable = true)
 |-- costo_total: double (nullable = true)

```

9.4 Transformaciones Analíticas Aplicadas

La capa Gold incorporó múltiples transformaciones orientadas al análisis del negocio y construcción de indicadores.

9.4.1 1. Agregaciones por Obra

Una de las principales transformaciones implementadas fue la generación de métricas de costos por obra.

Mediante operaciones GROUP BY y funciones de agregación de Spark se calcularon:

- Costos totales por obra.

- Cantidad de materiales consumidos.
- Distribución de consumos.
- Indicadores económicos generales.

Esta información permitió identificar:

- Obras con mayor nivel de consumo.
- Proyectos de mayor costo.
- Distribución económica de materiales.

09:37 (1 s)

```
from pyspark.sql.functions import sum, round, count, avg, countDistinct

gold_costo_por_obra = silver_final.groupBy("obra").agg(
    count("consumo_id").alias("cantidad_registros"),
    round(sum("cantidad"), 2).alias("cantidad_total_materiales"),
    round(avg("costo_total"), 2).alias("costo_promedio_consumo"),
    round(sum("costo_total"), 2).alias("costo_total_materiales")
).orderBy(
    "costo_total_materiales",
    ascending=False
)

print("Tabla Gold - Indicadores por obra")

display(gold_costo_por_obra)
```

(2) trabajos de Spark

gold_costo_por_obra: pyspark.sql.connect.dataframe.DataFrame = [obra: string, cantidad_registros: long ... 3 campos adicionales]

Tabla Gold - Indicadores por obra

Tabla	obra	cantidad_registros	cantidad_total_materiales	costo_promedio_consumo	costo_total_materiales
1	Escuela Técnica	50331	8024555.12	2894723.24	145694315373.5

Guardado de delta en GOLD

09:37 (2 s)

```
gold_costo_por_obra.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(f"{BASE}/gold/costo_por_obra")
```

(4) trabajos de Spark

Visualización de costos por obra

La siguiente visualización representa el costo total de materiales asociado a cada obra procesada dentro del pipeline.

El gráfico permite identificar rápidamente cuáles son los proyectos con mayor impacto económico, facilitando el análisis presupuestario y el monitoreo de costos operativos.

Esta visualización fue desarrollada utilizando las capacidades gráficas integradas de Azure Databricks orientadas al análisis gerencial y exploración de datos.



9.4.2 2. Generación de Ranking de Proveedores

También se construyeron métricas relacionadas con proveedores.

A partir de la información consolidada fue posible calcular:

- Proveedores con mayor participación.
- Cantidad de materiales suministrados.
- Impacto económico por proveedor.
- Ranking de costos asociados.

Estas métricas permiten realizar análisis estratégicos relacionados con abastecimiento y compras.

```

from pyspark.sql.functions import col, count, sum, round

gold_top_proveedores = silver_finel.filter(
    col("nombre_proveedor").isNotNull()
).groupBy(
    "nombre_proveedor",
    "ciudad",
    "provincia",
    "condicion_iva"
).agg(
    count("material_id").alias("cantidad_materiales"),
    round(sum("costo_total"), 2).alias("gasto_total_historico")
).orderBy(
    col("gasto_total_historico").desc()
)

print("Gold - Top proveedores por monto total hasta la fecha")

display(gold_top_proveedores)

```

(2) trabajos de Spark

gold_top_proveedores: pyspark.sql.connect.dataframe.DataFrame = [nombre_proveedor: string, ciudad: string ... 4 campos adicionales]

Gold - Top proveedores por monto total hasta la fecha

	nombre_proveedor	ciudad	provincia	condicion_iva	cantidad_materiales	gasto_total_historico
1	Eléctrica Rivadavia	La Plata	Buenos Aires	Responsable Inscripto	59830	219264881206.5

Guardado de delta en GOLD

```

gold_top_proveedores.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(f"{BASE}/gold/top_proveedores")

```

(4) trabajos de Spark

Visualización de proveedores con mayor gasto histórico

La siguiente visualización muestra los proveedores con mayor volumen económico acumulado dentro del histórico de consumos procesados.

El análisis permite identificar proveedores estratégicos y comprender la distribución del gasto asociado al abastecimiento de materiales.

Esta información resulta útil para procesos de negociación, evaluación comercial y análisis financiero de compras.



9.4.3 3. Generación de Ranking de Materiales

Además de las métricas relacionadas con proveedores, en la capa Gold también se construyeron indicadores orientados al análisis de materiales utilizados en las distintas obras.

A partir de la información consolidada fue posible calcular:

- Materiales con mayor impacto económico.
- Cantidad total consumida por material.
- Participación de cada material en el costo total.
- Cantidad de obras en las que fue utilizado cada material.
- Ranking de materiales más utilizados.

Estas métricas permiten identificar cuáles son los materiales más críticos dentro de los proyectos constructivos, facilitando:

- La planificación de compras.
- El control presupuestario.
- La detección de materiales de alto costo.
- El análisis de consumo histórico.
- La optimización del abastecimiento.

La tabla `top_materiales` se convirtió así en una de las principales fuentes analíticas del proyecto, permitiendo obtener una visión consolidada sobre el comportamiento económico y operativo de los materiales utilizados en las obras.

```

from pyspark.sql.functions import sum, count, countDistinct, avg

gold_top_materiales = sqlContext.sql("""
    select material,
    categoria_material
    from (
        select material,
        count(material) as cantidad_material,
        count(distinct obra) as cantidad_obra,
        count(distinct proveedor) as cantidad_proveedor,
        count(distinct distrito) as cantidad_distrito,
        count(distinct fecha) as cantidad_fecha
        from obra_materials
        group by material, categoria_material
    )
    order by cantidad_material desc
    limit 12
""")

print("Gold - Top materiales por impacto económico")

display(gold_top_materiales)

```

```

gold_top_materiales.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(f"{BASE}/gold/top_materiales")

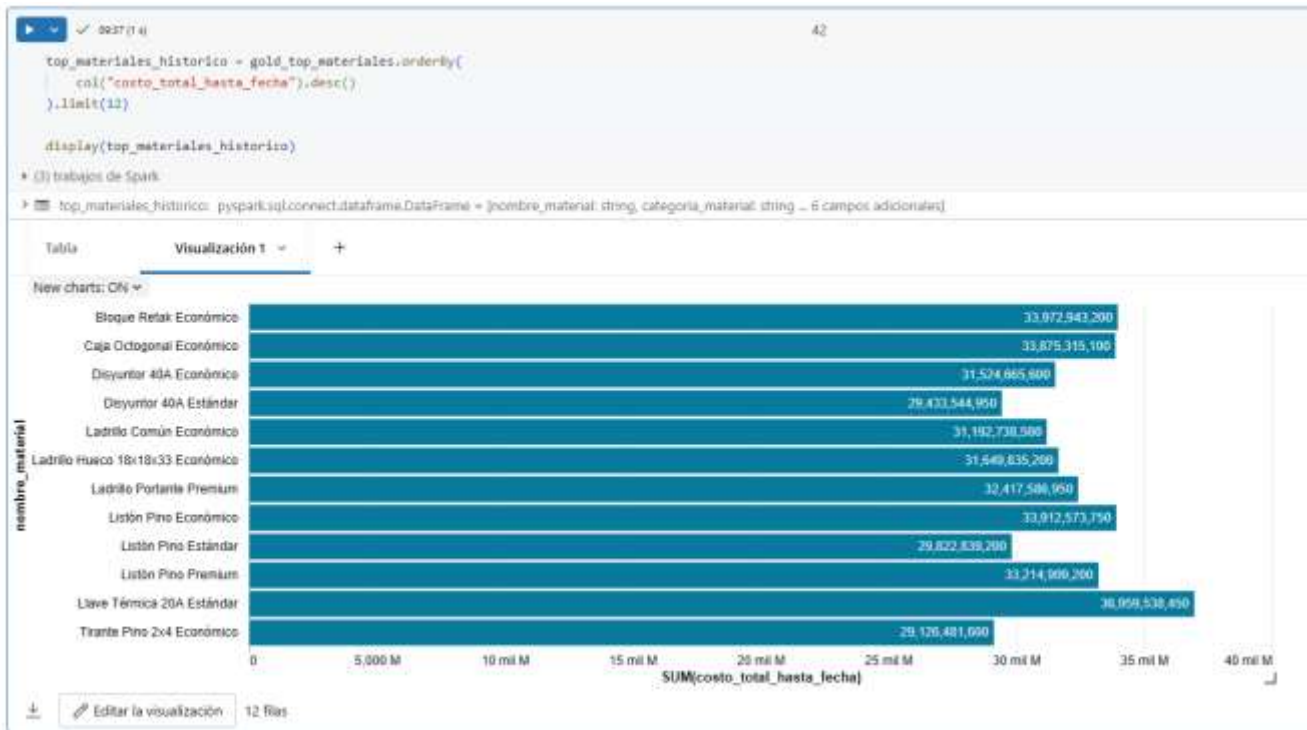
```

(5) trabajos de Spark

Visualización de materiales con mayor impacto económico

La siguiente visualización presenta un ranking histórico de los materiales con mayor impacto económico dentro de las obras procesadas. Las cuales para una visualización en el notebook se limitó a 12 materiales.

El análisis permite identificar cuáles son los materiales que generan mayores costos acumulados a lo largo del tiempo, facilitando el monitoreo financiero y la planificación de abastecimiento.



9.5 Construcción de Tablas Analíticas

Como resultado de las agregaciones realizadas en la capa Gold, se generaron tres tablas analíticas orientadas al análisis de negocio. Todas ellas toman como fuente principal la tabla consolidada de Silver:

/silver/consumos_materiales_proveedores

A partir de esta base validada e integrada, se construyeron salidas analíticas enfocadas en costos por obra, desempeño de proveedores e impacto económico de los materiales.

Tabla Gold	Objetivo
costo_por_obra	izar el costo total de materiales, cantidad de consumos, volumen consumido y costo promedio por cada obra
top_proveedores	tificar los proveedores con mayor gasto histórico acumulado, junto con su ubicación y condición impositiva
top_materiales	ectar los materiales con mayor impacto económico, volumen total consumido, cantidad de obras donde se usaron y proveedores asociados

Estas tablas fueron almacenadas en la capa Gold en formato Delta Lake, dentro de las rutas:

```
/gold/costo_por_obra  
  /gold/top_proveedores  
  /gold/top_materiales
```

La tabla top_materiales es especialmente importante porque permite analizar qué materiales concentran mayor costo acumulado dentro de las obras, facilitando la planificación de compras, la detección de materiales críticos y el control presupuestario.

9.6 Optimización de Tablas Gold

Debido a que las tablas Gold están orientadas a consumo analítico frecuente, se aplicaron procesos de optimización utilizando funcionalidades de Delta Lake.

Se ejecutó el comando:

9.6.1 Optimize

sobre las tablas generadas.

Esta optimización permitió:

- Compactar archivos pequeños.
- Mejorar tiempos de consulta.
- Reducir operaciones de lectura.
- Incrementar eficiencia en reporting.

La optimización resulta especialmente importante en tablas utilizadas por dashboards y consultas gerenciales.

9.6.2 Auditoría y Trazabilidad con DESCRIBE HISTORY

Sobre las tablas Gold también se utilizaron funcionalidades de auditoría mediante:

DESCRIBE HISTORY

Esto permitió consultar:

- Historial de versiones.
- Operaciones ejecutadas.
- Fechas de modificación.
- Procesos responsables.
- Información de escritura y optimización.

Estas capacidades fortalecieron la trazabilidad y el control operativo del pipeline.

9.7 Resultados Obtenidos

La capa Gold permitió transformar grandes volúmenes de datos operacionales en información analítica de alto valor para el negocio.

Entre los principales resultados obtenidos se destacan:

- Centralización de indicadores.
- Automatización de métricas.
- Reducción de procesamiento manual.
- Generación de información confiable.
- Preparación para visualización y reporting.

Además, las tablas Gold quedaron preparadas para ser utilizadas posteriormente por herramientas de Business Intelligence y análisis gerencial.

9.7.1 Importancia de la Capa Gold

La capa Gold representa el punto final del pipeline de ingeniería de datos implementado en el proyecto.

Es en esta etapa donde los datos finalmente adquieren valor estratégico para el negocio, permitiendo:

- Analizar costos.
- Evaluar proveedores.
- Detectar patrones de consumo.
- Optimizar decisiones operativas.
- Facilitar reporting ejecutivo.

La implementación de esta capa permitió completar exitosamente una arquitectura Medallion de punta a punta utilizando Azure Data Factory, Azure Databricks, Delta Lake y PySpark.

FLUJO DE TABLAS – CAPA GOLD

Transformación analítica y generación de tablas de negocio para consumo y reporting



10 Implementación del Pipeline Analítico en Azure Data Factory

Una vez finalizada la etapa de ingesta y almacenamiento de datos en la capa RAW, se desarrolló un pipeline de procesamiento analítico utilizando Azure Data Factory como herramienta principal de orquestación.

El objetivo de este pipeline fue automatizar completamente la ejecución de los notebooks desarrollados en Azure Databricks correspondientes a las capas Bronze, Silver y Gold de la arquitectura Medallion.

La implementación realizada permitió construir un flujo secuencial, escalable y desacoplado, garantizando que cada etapa del procesamiento se ejecute únicamente cuando la anterior haya finalizado correctamente.

10.1 Creación del Pipeline en Azure Data Factory

El proceso comenzó con la creación de un nuevo pipeline dentro de Azure Data Factory orientado exclusivamente a la ejecución de notebooks de Azure Databricks.

Dentro del pipeline se incorporaron actividades de tipo:

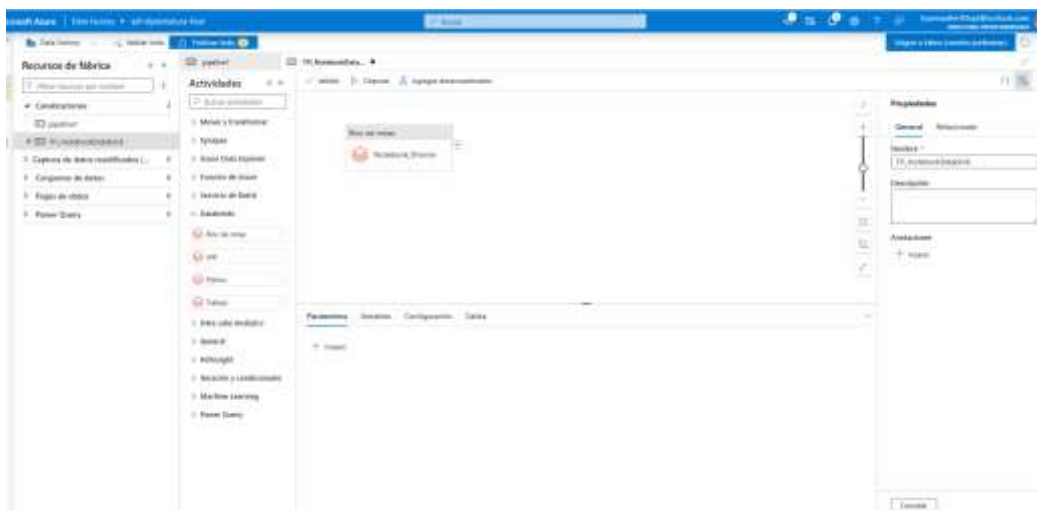
Databricks Notebook

Cada actividad representa la ejecución de un notebook específico dentro del workspace de Databricks.

Las actividades creadas fueron:

- Notebook_Bronce
- Notebook_Silver
- Notebook_Gold

Estas actividades fueron configuradas individualmente dentro del pipeline para ejecutar cada capa de la arquitectura Medallion.



10.2 Configuración del Servicio Vinculado con Databricks

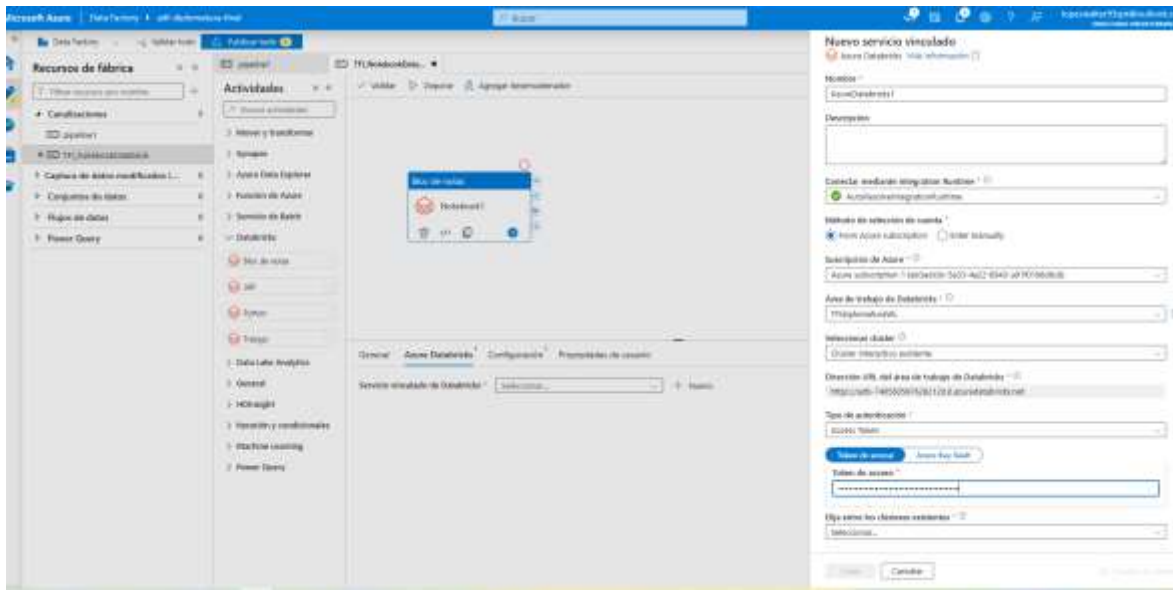
Para permitir la comunicación entre Azure Data Factory y Azure Databricks fue necesario configurar un Linked Service (Servicio Vinculado).

Esta configuración incluyó:

- Workspace de Azure Databricks.
- URL del entorno Databricks.
- Método de autenticación mediante Access Token.
- Selección del cluster de ejecución.
- Integration Runtime utilizado por ADF.

El Access Token utilizado fue generado previamente desde Azure Databricks y permitió establecer una conexión segura entre ambos servicios.

Gracias a este linked service, Azure Data Factory pudo ejecutar notebooks de Databricks directamente desde el pipeline de orquestación.



10.3 Configuración de los Notebooks

Una vez configurada la conexión con Databricks, cada actividad del pipeline fue asociada a su notebook correspondiente.

El pipeline quedó estructurado de la siguiente manera:

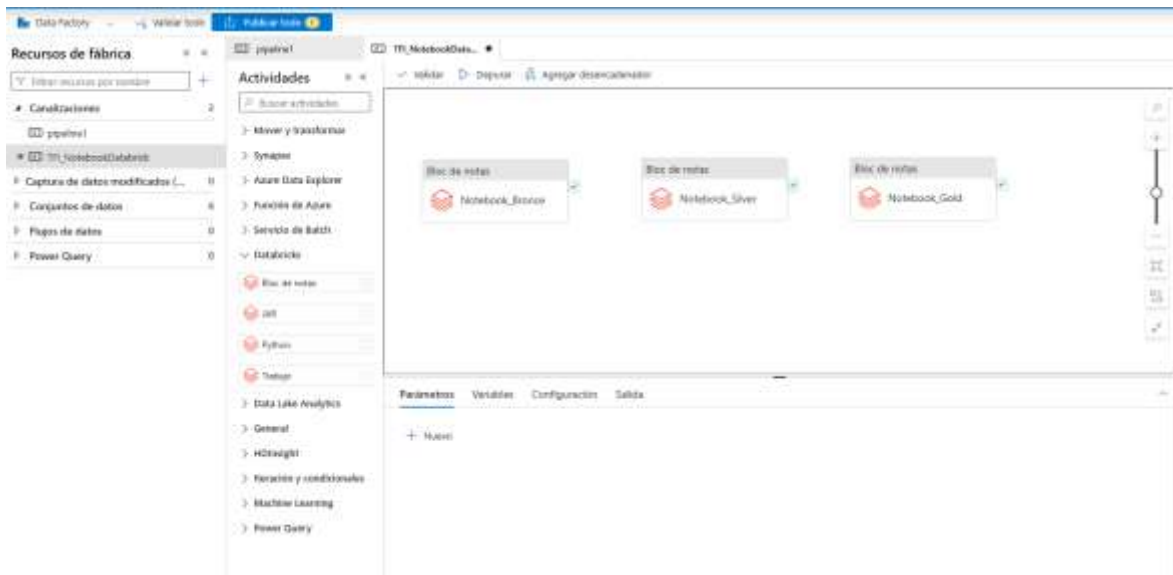
Notebook_Bronce
↓
Notebook_Silver

↓
Notebook_Gold

Cada notebook cumple una función específica dentro del procesamiento:

Notebook	Función
Notebook_Bronce	Lectura desde RAW y persistencia en Bronze
Notebook_Silver	Limpieza, validaciones y consolidación
Notebook_Gold	Generación de métricas y tablas analíticas

La ejecución secuencial permite respetar completamente el flujo lógico de la arquitectura Medallion.



10.4 Ejecución Secuencial del Pipeline

Uno de los aspectos más importantes de la implementación fue la configuración secuencial de las actividades.

Cada notebook fue conectado mediante dependencias de éxito, permitiendo que:

- Silver se ejecute únicamente cuando Bronze finalice correctamente.
- Gold se ejecute únicamente cuando Silver finalice correctamente.

Esta estrategia garantiza:

- Consistencia de datos.
- Correcto orden de procesamiento.
- Menor riesgo de errores.
- Mejor control operativo.

Además, evita la ejecución innecesaria de procesos posteriores en caso de fallas en etapas previas.

La orquestación secuencial implementada replica arquitecturas ETL/ELT utilizadas en entornos empresariales reales.



10.5 Validación y Depuración del Pipeline

Antes de publicar el pipeline se realizaron procesos de validación y depuración utilizando las herramientas integradas de Azure Data Factory.

Durante esta etapa se ejecutaron pruebas manuales mediante:

Debug

Esto permitió verificar:

- Correcta conexión con Databricks.
- Ejecución exitosa de notebooks.
- Dependencias entre actividades.
- Correcto funcionamiento del linked service.
- Tiempos de ejecución.
- Estado general del pipeline.

La depuración permitió identificar y corregir configuraciones antes de la publicación definitiva del flujo.

TFI_NotebookData...

Validar Depurar Agregar desencadenador

Bloc de notas Notebook_Bronce

Bloc de notas Notebook_Silver

Bloc de notas Notebook_Gold

Parámetros Variables Configuración **Salida**

Id. de ejecución de canalización: d3065544-7a32-4df5-9f0c-afed4bd7c128

Estado de la canalización: Correcto

All status [Supervisión en métricas de Azure](#) [Exportar a CSV](#)

Mostrando elementos del 1 al 3 de un total de 3

Nombre de actividad	Estado de ...	Tipo d...	Inicio de la ejecu...	Duraci...	Entorno de ejecución de integración	Pre
Notebook_Gold	Correcto	Bloc de notas	5/7/2026, 11:10:22 AM	1m 3s	AutoResolveIntegrationRuntime (Brazil South)	
Notebook_Silver	Correcto	Bloc de notas	5/7/2026, 11:08:01 AM	2m 20s	AutoResolveIntegrationRuntime (Brazil South)	
Notebook_Bronce	Correcto	Bloc de notas	5/7/2026, 11:06:13 AM	1m 44s	AutoResolveIntegrationRuntime (Brazil South)	

10.6 Publicación del Pipeline

Una vez completadas las validaciones y pruebas de ejecución, el pipeline fue publicado dentro del entorno de Azure Data Factory.

La publicación permitió:

- Activar el pipeline en el entorno productivo.
- Habilitar triggers automáticos.
- Guardar configuraciones definitivas.
- Disponibilizar el flujo para ejecuciones programadas.

Este paso representa la puesta en producción del proceso analítico automatizado.

The screenshot displays the Databricks interface. On the left, a pipeline named 'TFI_NotebookData...' is shown with three stages: 'Notebook_Gold', 'Notebook_Silver', and 'Notebook_Bronze'. The 'Salida' (Output) tab is active, showing the pipeline's execution history. The history table indicates that all three stages completed successfully on 5/7/2024.

On the right, the 'Publicar todo' (Publish All) dialog is open. It shows the pipeline's name 'TFI_NotebookData...' and its status as '(Nuevo)' (New). The dialog includes a 'Publicar' (Publish) button and a 'Cancelar' (Cancel) button.

10.7 Configuración del Trigger Automático

Posteriormente se configuró un trigger programado para automatizar la ejecución diaria del pipeline.

El trigger creado fue:

TFI_Notebook_Medallions

La configuración establecida fue:

Parámetro	Configuración
Tipo	Programado
Frecuencia	Diaria
Hora de ejecución	10:00 AM
Zona horaria	Buenos Aires (UTC-3)

La finalidad de este trigger es ejecutar automáticamente el pipeline todos los días luego de finalizar el proceso de ingesta de datos.

Nuevo desencadenador

Nombre *

TFI_Notebook_Medallions

Descripción

Tipo *

Programar

Fecha de inicio * ⓘ

5/8/2026, 10:00:00 AM

Zona horaria * ⓘ

Buenos Aires (UTC-3)

Periodicidad * ⓘ

Cada 1

Días

> Opciones avanzadas de periodicidad

☐ Especificar una fecha de finalización

Anotaciones

+ Nuevo

Iniciar desencadenador ⓘ

☒ Iniciar desencadenador al crea

Aceptar Cancelar

10.8 Relación entre Ingesta y Procesamiento

La estrategia implementada contempla dos etapas claramente separadas:

10.8.1 Pipeline de Ingesta (09:30 AM)

Responsable de:

- Lectura de archivos.
- Movimiento hacia RAW.
- Conversión de formatos.
- Actualización de datasets base.

10.8.2 Pipeline Analítico (10:00 AM)

Responsable de:

- Procesamiento Bronze.
- Validaciones Silver.
- Generación de tablas Gold.
- Optimización Delta.
- Construcción de métricas analíticas.

La diferencia horaria entre ambos procesos garantiza que los datos ya se encuentren disponibles antes de comenzar las transformaciones analíticas.

10.9 Monitoreo de Ejecuciones

Azure Data Factory permitió monitorear en tiempo real el estado de ejecución de cada notebook dentro del pipeline.

La interfaz de monitoreo permitió visualizar:

- Estado de cada actividad.
- Hora de inicio.
- Tiempo de duración.
- Resultado de ejecución.
- Runtime utilizado.
- Estado general del pipeline.

Esto aporta importantes capacidades de:

- Auditoría.
- Seguimiento operativo.
- Monitoreo de errores.
- Trazabilidad de procesos.

10.10 Beneficios de la Orquestación Implementada

La solución implementada permitió construir un flujo de trabajo completamente automatizado y alineado con buenas prácticas modernas de ingeniería de datos.

10.10.1 Beneficios Técnicos

- Integración nativa entre ADF y Databricks.
- Ejecución secuencial controlada.



- Escalabilidad del procesamiento.
- Automatización completa.
- Monitoreo centralizado.

10.10.2 Beneficios Operativos

- Eliminación de ejecuciones manuales.
- Mayor estabilidad del pipeline.
- Reducción de errores humanos.
- Disponibilidad automática de información analítica.
- Mejor control del flujo de procesamiento.

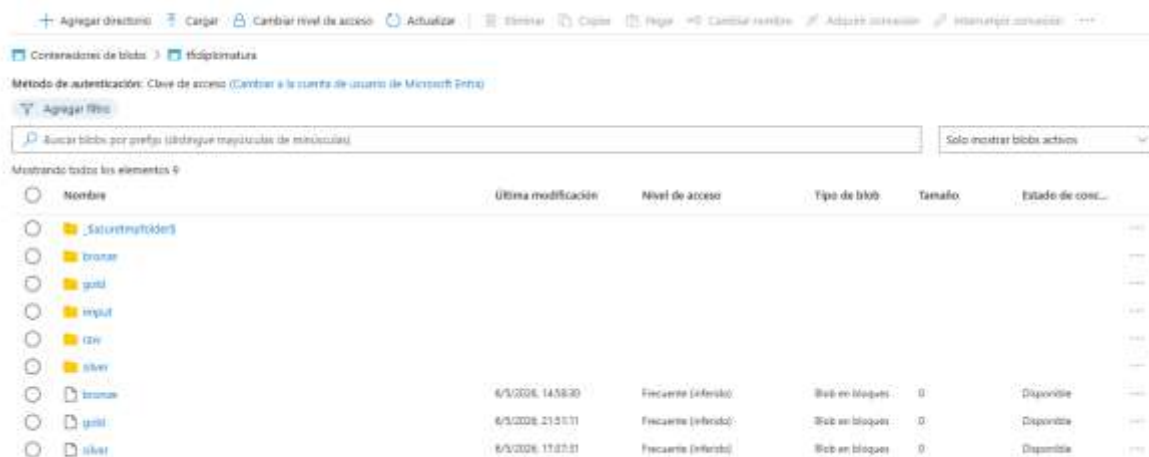
La implementación realizada permitió simular un entorno empresarial real de procesamiento ETL/ELT automatizado utilizando tecnologías cloud modernas del ecosistema Microsoft Azure.

Resultados del Proyecto		
Indicador	Descripción	Resultado
 Registros procesados	Cantidad total de registros procesados de todas las fuentes.	500.000+
 Fuentes integradas	Cantidad de fuentes de datos heterogéneas integradas.	3
 Capas implementadas	Implementación completa de la arquitectura Medallion.	4 (RAW, Bronze, Silver, Gold)
 Pipelines implementados	Pipelines desarrollados en Azure Data Factory.	2
 Triggers configurados	Automatización de ejecución programada de pipelines.	2
 Tablas Gold generadas	Tablas analíticas generadas para el análisis de negocio.	4
 Formatos utilizados	Formatos de archivos y tecnologías utilizadas en el procesamiento.	CSV, JSON, Parquet, Delta Lake

11 Vista del contenedor Blob Storage

La siguiente imagen presenta la estructura general del contenedor principal utilizado dentro de Azure Blob Storage para el desarrollo del proyecto. En este entorno se organizaron las distintas capas de almacenamiento siguiendo la arquitectura Medallion implementada durante el trabajo.

Dentro del contenedor tfdiplomatura se pueden observar las carpetas correspondientes a las capas raw, bronze, silver y gold, además de la carpeta input, utilizada inicialmente para la carga de archivos fuente provenientes de las distintas fuentes de datos integradas. Esta organización permitió mantener una separación clara entre cada etapa del procesamiento, facilitando la trazabilidad, el control de versiones y la escalabilidad del pipeline de datos desarrollado.



12Compartir Usuario

<hidden for security>

13 Grupo de Recurso

El grupo se llama **TFI_Diplomatura_DE_WL** y funciona como una “carpeta lógica” donde Azure agrupa todos los servicios relacionados con tu trabajo final.

Dentro de ese grupo aparecen todos los recursos que creaste para el proyecto:

Recurso	Qué es	Para qué lo usás
adf-diplomatura-final	Servicio de Azure Data Factory	Crear pipelines, triggers y mover datos
free-sql-db-1954703	Base de datos SQL	Guardar tablas y datos procesados
sqldiplomatura	SQL Server	Servidor donde vive la base SQL
storagediplomatura2026	Storage Account	Guardar CSV, JSON, capas bronze/silver/gold
TFIdiplomaturaWL	Azure Databricks	Procesamiento con PySpark y Delta Lake

La idea del grupo de recursos es:

- administrar todo junto,
- controlar costos,
- borrar todo el proyecto de una sola vez si fuera necesario,
- y mantener organizado el entorno.

El cual contiene:

- Suscripción → la cuenta de Azure que está pagando/usando los recursos.
- Ubicación: Brazil South → región física donde están desplegados los servicios.
- Implementaciones: 4 Correcta → despliegues/configuraciones realizadas sin errores.

14 Importante pipeline TFI_ingesta_InputRAW de Azure Database

Copiar el siguiente código en el editor de consulta donde se aloja el sql PROVEDORES para que se genere el link de permiso en ADF

```
ALTER ROLE db_owner ADD MEMBER [adf-diplomatura-final];
```

15 Carpetas de Databrick



Nombre	Tipo	Propietario	Creación
Capa RAW	Cuaderno	WALTER LOPEZ	06 may 2026, 14:40
Capa GOLD	Cuaderno	WALTER LOPEZ	06 may 2026, 22:00
Capa SILVER	Cuaderno	WALTER LOPEZ	06 may 2026, 15:23

Token: <hidden for security>

15.1 Link de Notebooks Medallion

[<hidden for security>](https://drive.google.com/)

En dicho link están guardados los archivos ipynb

16 Conclusiones y Reflexiones Finales

El desarrollo del presente Trabajo Final Integrador permitió construir una solución completa de ingeniería de datos utilizando herramientas modernas del ecosistema Microsoft Azure y procesamiento distribuido con Azure Databricks.

A lo largo del proyecto fue posible diseñar e implementar un pipeline de datos end-to-end capaz de integrar múltiples fuentes heterogéneas, automatizar procesos de ingesta, aplicar transformaciones analíticas y generar información confiable orientada a la toma de decisiones.

Uno de los aspectos más importantes del trabajo fue poder replicar un escenario similar al de una empresa constructora real, donde diariamente se generan grandes volúmenes de información relacionados con consumos de materiales, proveedores, costos y obras. Este enfoque permitió comprender cómo los problemas de calidad de datos, descentralización de información y procesos manuales impactan directamente en la operación y en la capacidad analítica de una organización.

La implementación de la arquitectura Medallion permitió estructurar el procesamiento de datos de manera clara y ordenada, separando cada etapa según su responsabilidad dentro del ciclo de vida de la información. La utilización de las capas RAW, Bronze, Silver y Gold permitió desacoplar completamente los procesos de ingesta, validación y análisis, logrando una arquitectura más mantenible, escalable y alineada con buenas prácticas actuales de ingeniería de datos.

Durante el desarrollo del proyecto también surgieron distintos desafíos técnicos que resultaron fundamentales para el aprendizaje práctico. Uno de los principales desafíos fue la integración de múltiples formatos de datos dentro de una misma arquitectura. Trabajar simultáneamente con archivos CSV, JSON, bases SQL, Parquet y tablas Delta implicó comprender diferentes estructuras, mecanismos de lectura y estrategias de procesamiento.

Otro desafío importante estuvo relacionado con la calidad de los datos. Durante las pruebas realizadas aparecieron registros duplicados, valores nulos, inconsistencias entre datasets y problemas de tipado. Esto obligó a implementar validaciones, filtros y mecanismos de limpieza dentro de la capa Silver para garantizar que únicamente información consistente llegara a las tablas analíticas finales.

La integración entre Azure Data Factory y Azure Databricks también representó una parte importante del trabajo. La configuración de linked services, access tokens, clusters y triggers permitió comprender cómo se construyen procesos ETL/ELT automatizados dentro de entornos cloud modernos. La orquestación secuencial de notebooks mediante Azure Data Factory permitió además simular un flujo productivo real, donde cada etapa depende correctamente de la finalización exitosa de la anterior.

Uno de los aprendizajes más relevantes del proyecto fue comprender la importancia de la automatización y la trazabilidad dentro de una arquitectura de datos. La implementación de triggers programados, el uso de Delta Lake y funcionalidades como OPTIMIZE o DESCRIBE HISTORY permitieron incorporar capacidades avanzadas de versionado, auditoría y optimización que resultan fundamentales en escenarios empresariales reales.

Desde el punto de vista analítico, el proyecto permitió transformar grandes volúmenes de datos operacionales en información útil para el negocio. Las tablas Gold generadas posibilitan analizar costos por obra, evaluar proveedores estratégicos y detectar materiales con mayor impacto económico, facilitando procesos de planificación, abastecimiento y control presupuestario.

Si bien la solución implementada cumple con los objetivos planteados inicialmente, el proyecto deja abierta la posibilidad de incorporar futuras mejoras y extensiones. Entre ellas podrían incluirse integraciones con APIs reales de materiales y precios, procesamiento en tiempo real mediante streaming, integración con Power BI para visualización ejecutiva o incluso mecanismos avanzados de gobierno y catalogación de datos.

Finalmente, el desarrollo de este trabajo permitió consolidar conocimientos fundamentales de ingeniería de datos moderna, aplicando de forma práctica conceptos relacionados con procesamiento distribuido, arquitecturas cloud, pipelines automatizados y almacenamiento analítico. Más allá de los resultados técnicos obtenidos, el proyecto permitió comprender cómo diseñar soluciones escalables, organizadas y preparadas para escenarios reales de procesamiento masivo de datos.